THU ASP.NET · WEEK 07

通過驗證，
才值得真正留下來。

延續小安的校園活動報名：這週把合法輸入交給 EF Core，寫進 SQL Server，再從資料庫讀回畫面。

下載學生任務包 | 查看預期成品 Demo ↗ | PDF 講義

BEFORE → AFTER

成功訊息只存在這次回應；停止網站後就消失。

第六週

合法資料通過 ModelState，Razor 顯示摘要。

重新啟動後，剛才的報名沒有留下紀錄。

第七週

合法資料寫進資料庫，重新啟動後仍能查到。

畫面、資料表、重啟後查詢三處一致。

TODAY · 3 × 50 MIN

今天不貪多：
建立、連線、保存、讀回。

01

資料形狀

分清 Input Model、Entity 與 DbContext。

實作 15 分鐘

02

資料表

設定連線，使用 Migration 建立資料表。

實作 20 分鐘

03

保存證據

合法才寫入，查回列表並重啟驗證。

實作 20 分鐘

先完成可驗收的一條路，再談完整 CRUD

MENTAL MODEL · END TO END

資料不是跳進資料庫，它會依序經過八站。



下一站 →

第一站：小安在瀏覽器輸入課堂假資料。

哪一站出錯，就打開哪一站檢查

PERSISTENCE · LIFETIME

程式記得一陣子，
資料庫才會跨越重啟。

1

區域變數

這次方法結束後就不再使用。

2

PageModel 屬性

服務這次請求與回應；下一次重新建立。

3

資料庫資料列

寫入成功後，網站停止與重啟仍可查詢。

本週的驗收不是「畫面出現成功」；而是「新增一筆、資料表查得到、重啟後仍在」。

MODEL · SEPARATE RESPONSIBILITIES

輸入表單與資料表，不必是同一個 C# 類別。

RegistrationInput

接住使用者這次輸入。

- 有 Required、EmailAddress
- 可保留 UI 專用欄位
- 不代表資料表已保存

Registration Entity

代表資料庫的一筆報名。

- 有主鍵 Id
- 保存通過驗證的值
- 可加入 CreatedAt

A 接表單：Entity

B 存資料：Input

C 兩者同一個

D Input 接、Entity 存

快問：哪個描述正確？

Input 是入口；Entity 是保存形狀

ENTITY · ONE ROW

一個 Registration 物件，對應資料表的一列。

```
public class Registration
{
    public int Id { get; set; }
    public string DisplayName { get; set; } = "";
    public string Email { get; set; } = "";
    public string Session { get; set; } = "";
    public string? Note { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

先只看三件事

Id：主鍵，用來分辨每一筆。

欄位：要保存的值。

CreatedAt：由後端產生，不讓表單決定。

Entity 先保持簡單，不在本週加入關聯

DbContext 不是資料庫，
它是程式與資料庫之間的工作窗口。




```
public class CampusActivityContext : DbContext
{
    public CampusActivityContext(
        DbContextOptions<CampusActivityContext> options)
        : base(options) { }

    public DbSet<Registration> Registrations =>
        Set<Registration>();
}
```

DbSet 是從程式操作 Registration 資料表的入口。

暫停講解 · 學生實作

PAUSE & PRACTICE 00

15 分鐘

先建立 Entity 與 DbContext，不急著連線。

查看本週完成條件 Demo ↗

學生現在完成

1. 新增 `Models/Registration.cs`。
2. 建立 Id、四個保存欄位與 CreatedAt。
3. 新增 `Data/CampusActivityContext.cs`。
4. 建立 `DbSet<Registration>`。

完成條件

- 專案 Build 沒有紅線。
- 能指出哪個類別代表一列。
- 能指出哪個屬性是主鍵。
- 能說明 Input Model 尚未被取代。

卡住先查：namespace 與 using 是否對到專案。

第 8 分鐘：先完成 Entity，再接 DbContext。

快學生：畫出類別、DbSet、資料表三者關係。

- [ ] 我已完成 Entity 與 DbContext，並能口頭指出主鍵與保存欄位。

時間到：兩人互指 Entity、DbContext 與 Input Model

CHECKPOINT 01 · POINT AND EXPLAIN

不要背定義，
直接在自己的專案指出位置。

1

一筆資料

指出 Registration，說明哪五個值會保存。

2

主鍵

指出 Id，說明每筆資料需要不同識別。

3

操作入口

指出 Registrations DbSet，說明它不是資料表本身。

先說清楚資料形狀，再開始連線

CONFIGURATION · PROGRAM.CS

先把 DbContext 放進服務容器，PageModel 才能取得它。

```
using Microsoft.EntityFrameworkCore;
using Week07Registration.Data;

builder.Services.AddRazorPages();

builder.Services.AddDbContext<CampusActivityContext>(
    options => options.UseSqlServer(
        builder.Configuration.GetConnectionString(
            "CampusActivity")));
```

讀法

建立 `CampusActivityContext` 時，使用 SQL Server provider。

真正的目的地從設定檔讀取，不把連線字串散落在 PageModel。

服務註冊回答「怎麼建立 Context」

CONNECTION STRING · DESTINATION

連線字串告訴程式去哪裡，不是拿來保存帳密的筆記。

```
"ConnectionStrings": {
  "CampusActivity":
    "Server=.\\SQLEXPRESS;Database=CampusActivityDb;
     Trusted_Connection=True;TrustServerCertificate=True"
}
```

課堂本機：使用 Windows 驗證與自己的 SQL Server Express 執行個體。

實際部署：秘密放環境變數或 Secret Manager，不提交帳密到 Git。

執行個體名稱不同時，先查自己的環境

MIGRATION · SCHEMA PLAN

Migration 是資料表結構的變更計畫，不是報名資料本身。

Entity

目前 C# 想要的資料形狀

Migration

把形狀轉成可追蹤的結構變更

Update

把尚未套用的變更送到資料庫

Table

SQL Server 出現 Registrations

Migration 檔要看懂並提交；資料庫是否套用仍需實際查證。

TOOLS · TWO COMMANDS

命令只做兩步：
產生計畫，再更新資料庫。

Visual Studio 套件管理主控台

```
Add-Migration CreateRegistrations
Update-Database
```

Default project 要選目前 Web 專案。

終端機備援

```
dotnet ef migrations add CreateRegistrations
dotnet ef database update
```

在含有 .csproj 的資料夾執行。

看到 Build succeeded 只代表工具開始順利；仍要到 SSMS 查資料庫與欄位。

VERIFY · SQL SERVER

不要只相信終端機成功訊息，打開 SSMS 看實際資料表。

```
USE CampusActivityDb;
GO
SELECT TOP (10)
    Id, DisplayName, Email,
    Session, Note, CreatedAt
FROM dbo.Registrations
ORDER BY Id DESC;
```

建立後先核對

- CampusActivityDb 存在
- dbo.Registrations 存在
- 欄位名稱與型別合理
- 目前 0 筆是正常的

資料表存在，不代表已經存入報名

暫停講解 · 學生實作

PAUSE & PRACTICE 01

20 分鐘

完成連線與 Migration，親眼看到資料表。

查看本週完成條件 Demo ↗

學生現在完成

1. 安裝 EF Core SQL Server 與 Tools / Design 套件。
2. 設定 Program.cs 與 ConnectionStrings。
3. 建立 `CreateRegistrations` Migration。
4. 執行 Update-Database，再用 SSMS 查欄位。

完成條件

- Migration 檔存在且能讀出 CreateTable。
- SQL Server 有 CampusActivityDb。
- dbo.Registrations 欄位完整。
- 查詢可執行，目前 0 筆不算錯。

卡住先查：錯在套件、Context、連線或 Migration 哪一站。

第 10 分鐘：尚未產生 Migration 者先停下核對連線。

快學生：比對 Migration 欄位與 Entity。

- [ ] 我已用 SSMS 看到 dbo.Registrations 與正確欄位。

時間到：展示資料表結構，不展示真實個資

CHECKPOINT 02 · FOUR STATIONS

資料表沒出現時，
依四站定位，不直接重建整個專案。

1 套件
版本與目標框架相容？

→

2 Context
Program 有註冊？

→

3 連線
執行個體與 DB 名稱？

→

4 Migration
有產生且已套用？

> 完整記錄：執行哪個命令、預期什麼、實際錯誤第一行、在哪一站停止。

SAVE GATE · VALID FIRST

資料庫不該替表單驗證收爛攤子。
ModelState 失敗就立刻返回。

```
public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        await LoadRegistrationsAsync();
        return Page();
    }

    // 只有通過驗證才建立 Entity 與保存
}
```

回到同一頁時仍需重新載入列表，避免畫面只剩表單、原有資料消失。

失敗路線要證明資料列數沒有增加

MAPPING · CHOOSE WHAT TO SAVE

不要把整個輸入物件直接當資料列；明確選擇要保存的欄位。

```
var registration = new Registration
{
    DisplayName = Input.DisplayName,
    Email = Input.Email,
    Session = Input.Session,
    Note = Input.Note,
    CreatedAt = DateTime.UtcNow
};
```

後端負責的值

Id：由資料庫產生。

CreatedAt：由後端產生。

不保存：成功訊息、確認勾選等 UI 狀態。

EF CORE · TRACK THEN COMMIT

Add 先讓 Context 追蹤；SaveChangesAsync 才送到資料庫。

```
_db.Registrations.Add(registration);
await _db.SaveChangesAsync();

TempData["SavedMessage"] = "報名已保存";
return RedirectToPage();
```

Add：登記「這是一筆要新增的資料」。

SaveChangesAsync：真正執行 INSERT；成功後 Redirect，重新整理不會重送同一筆 POST。

QUERY · READBACK

保存後再查回，畫面才有資料庫證據。

```
public List<Registration> Registrations { get; set; } = [];

public async Task OnGetAsync()
{
    await LoadRegistrationsAsync();
}

private async Task LoadRegistrationsAsync()
{
    Registrations = await _db.Registrations
        .AsNoTracking()
        .OrderByDescending(x => x.Id)
        .Take(10)
        .ToListAsync();
}
```

本週只取最近 10 筆

避免一次載入全部資料。

列表只顯示課堂假資料。

先不教搜尋、分頁、編輯與刪除。

Query 的結果交給 Razor 顯示

暫停講解 · 學生實作

PAUSE & PRACTICE 02

20 分鐘

完成合法保存、列表讀回與重啟驗證。

查看本週完成條件 Demo ↗

學生現在完成

1. 注入 Context，加入保存前 ModelState 閘門。
2. 把 Input 明確轉成 Registration。
3. Add、SaveChangesAsync、RedirectToPage。
4. OnGetAsync 查最近 10 筆並顯示列表。

完成條件

- 空白 / 錯誤 Email：不新增資料列。
- 合法假資料：只新增 1 筆。
- 畫面與 SSMS 內容一致。
- 停止再啟動，該筆仍能查到。

卡住先查：先比資料列數，判斷是沒寫入或沒顯示。

第 10 分鐘：先讓一筆合法資料完成端到端。

快學生：重新整理成功頁，確認不重複新增。

☐ 我已完成失敗不新增、成功新增一筆、重啟後仍在三項證據。

時間到：兩人交換測試，不替對方直接修改

FINAL BOSS · DEBUG WITH EVIDENCE

小安說「按下保存了」，
你要用證據判斷卡在哪一站。

現象	先查哪裡	通過證據
錯誤 Email 也新增	ModelState 閘門順序	資料列數不增加
成功訊息有，但 SSMS 沒資料	SaveChangesAsync 與連線目的地	同一 DB 出現 1 筆
SSMS 有資料，畫面沒列表	OnGetAsync / Razor foreach	畫面與查詢一致
重新整理又多一筆	RedirectToPage	GET 不重送 POST

重現 → 定位 → 最小修改 → 重跑同一組測試

MISSION COMPLETE · WEEK 07

你完成的不是一句成功訊息，而是一條可重新查證的保存路線。

形狀清楚

能區分 Input Model、Entity、DbContext。

資料表可驗

Migration、SSMS 與 Entity 欄位一致。

保存有證據

失敗不新增、成功只一筆、重啟後仍在。

本週邊界：只做 Create＋Read；編輯、刪除、關聯、登入權限與正式部署留到後續課程。

下載學生任務包

再看一次完成條件 ↗

下一步：用路由與 Id 操作指定資料